Vidyavardhini's College of Engineering &Technology
Department of Computer Engineering
Academic Year: 2026-27

Name of the Student: Mohammed Shahiduddin Shahalam Shaikh
Class and Div and Roll No: TE-2/71
Experiment No.6
Aim: Implementation of a TCP Client–Server Application Using Socket Programming
Date of Performance: 18/08/2026
Date of Submission: 27/08/2026

Experiment No 6

AIM: Implementation of a TCP Client–Server Application Using Socket

THEORY:

Socket Programming is a method of communication between two devices over a network using sockets. A socket is one endpoint of a two-way communication channel between two applications. Socket programming enables processes running on different computers to exchange data over a network.

The Transmission Control Protocol (TCP) is a reliable, connection-oriented transport layer protocol. It ensures that data is delivered accurately, in sequence, and without loss. Before transmitting data, TCP establishes a connection between the client and the server using a process known as the three-way handshake. Once communication is complete, the connection is terminated gracefully.

In a TCP Client–Server architecture, two programs participate in communication:

- Server: The server waits for incoming client requests on a specific port. It accepts connection requests, receives data from clients, processes the request, and sends an appropriate response.
- Client: The client initiates communication by connecting to the server's IP address and port number. After establishing the connection, it sends a request and waits for the server's response.

Working of TCP Client–Server Communication

1. The server creates a socket.
2. The server binds the socket to an IP address and port number.
3. The server listens for incoming client requests.

4. The client creates its socket.
5. The client sends a connection request to the server.
6. The server accepts the connection.
7. Data is exchanged between the client and server using send() and recv() (or write() and read()) functions.
8. After communication is complete, both the client and server close their sockets.

Common Socket Functions

Function	Description
socket()	Creates a new socket.
bind()	Assigns an IP address and port number to the server socket.
listen()	Places the server in listening mode for incoming connections.
accept()	Accepts a connection request from a client.
connect()	Establishes a connection from the client to the server.
send()	Sends data through the socket.
recv()	Receives data from the socket.
close()	Closes the socket and terminates the connection

PROGRAM FOR TCP USING C++:

Client Side Program:

```
#include <iostream>
#include <winsock2.h>
#include <ws2tcpip.h>
#pragma comment(lib, "ws2_32.lib")
using namespace std;

int main() {
    WSADATA wsaData;
    WSAStartup(MAKEWORD(2, 2), &wsaData);

    SOCKET serverSocket = socket(AF_INET, SOCK_STREAM, 0);

    sockaddr_in serverAddr;
    serverAddr.sin_family = AF_INET;
    serverAddr.sin_addr.s_addr = INADDR_ANY;
    serverAddr.sin_port = htons(8080);

    bind(serverSocket, (sockaddr*)&serverAddr, sizeof(serverAddr));
    listen(serverSocket, 5);

    cout << "Server is waiting for client..." << endl;

    SOCKET clientSocket = accept(serverSocket, NULL, NULL);
    cout << "Client connected!" << endl;

    char buffer[1024];
    while (true) {
        memset(buffer, 0, sizeof(buffer));
        int bytesReceived = recv(clientSocket, buffer, sizeof(buffer), 0);
```

Server Side Program:

```
#include <iostream>
#include <winsock2.h>
#include <ws2tcpip.h>
#pragma comment(lib, "ws2_32.lib")
using namespace std;

int main() {
    WSADATA wsaData;
    WSAStartup(MAKEWORD(2, 2), &wsaData);

    SOCKET clientSocket = socket(AF_INET, SOCK_STREAM, 0);

    sockaddr_in serverAddr;
    serverAddr.sin_family = AF_INET;
    serverAddr.sin_port = htons(8080);
    serverAddr.sin_addr.s_addr = inet_addr("127.0.0.1");

    connect(clientSocket, (sockaddr*)&serverAddr, sizeof(serverAddr));
    cout << "Connected to server!" << endl;

    char buffer[1024];
    while (true) {
        string message;
        cout << "Client: ";
        getline(cin, message);

        send(clientSocket, message.c_str(), message.size(), 0);
        memset(buffer, 0, sizeof(buffer));
        int bytesReceived = recv(clientSocket, buffer, sizeof(buffer), 0);
```

```
        if (bytesReceived <= 0) {
            cout << "Client disconnected." << endl;
            break;
        }

        cout << "Client: " << buffer << endl;

        string reply;
        cout << "Server: ";
        getline(cin, reply);

        send(clientSocket, reply.c_str(), reply.size(), 0);
    }

    closesocket(clientSocket);
    closesocket(serverSocket);
    WSACleanup();
    return 0;
}
```

```
        if (bytesReceived <= 0) {
            cout << "Server disconnected." << endl;
            break;
        }

        cout << "Server: " << buffer << endl;
    }

    closesocket(clientSocket);
    WSACleanup();
    return 0;
}
```

OUTPUT:

```
Connected to server!
Client: Hello Server!
Server: Hi Client
Client:
```

```
Server is waiting for client...
Client connected!
Client: Hello Server!
Server: Hi Client
```

Conclusion:

Using TCP socket programming, a reliable client–server application can be implemented in which the client establishes a connection with the server, exchanges data, and terminates the connection after successful communication. The use of TCP ensures reliable, ordered, and error-free data transmission between the communicating devices.